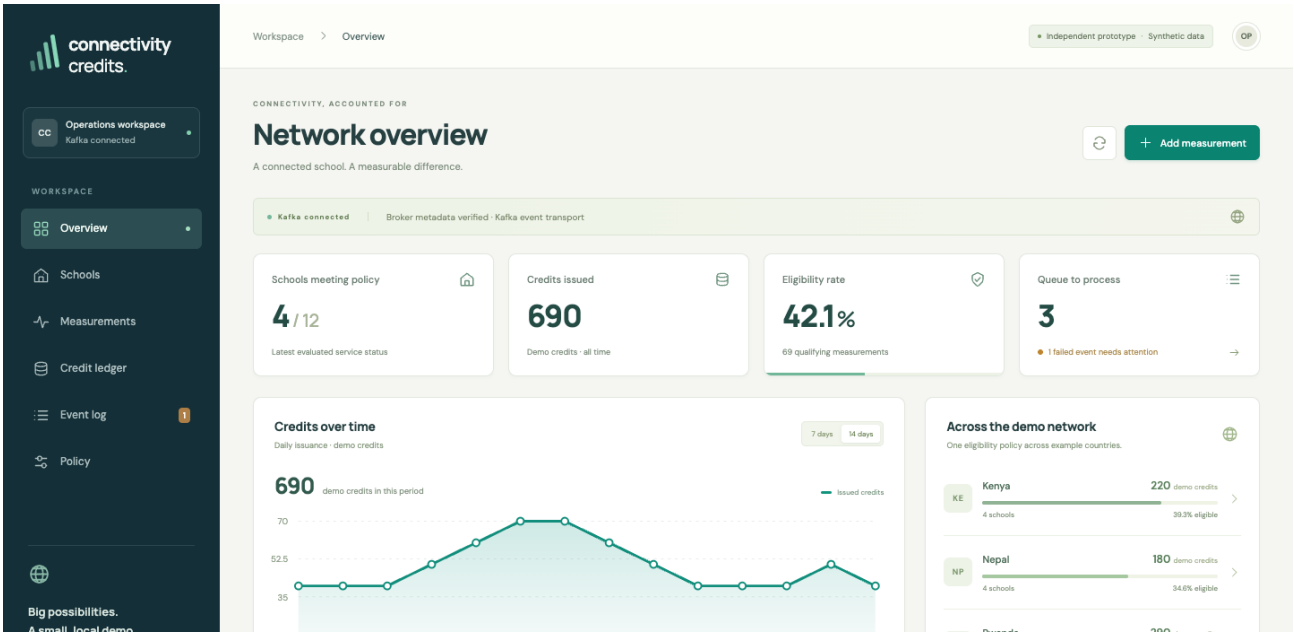
TECHNICAL OVERVIEW

From connectivity readings to an auditable credit ledger

A working demonstration of how school internet measurements can be checked against a policy and converted into traceable demo credits.

Prepared by Ankit Pasayat for review alongside the Connectivity Credits backend consultancy application. The focus is the engineering workflow: reliable processing, clear decisions and safe recovery from failure.



Working console with synthetic school readings, credit totals and live Kafka status.

What the prototype demonstrates

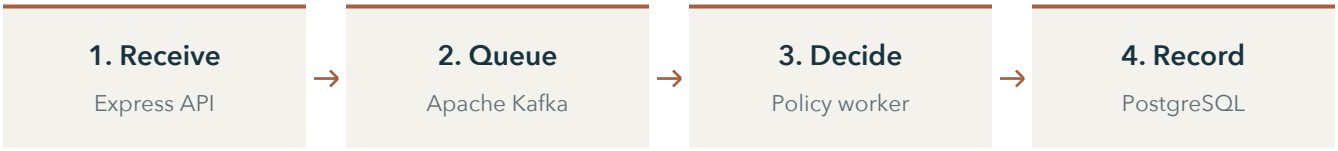
Receive a measurement, **check** five eligibility conditions, **record** a decision, and **issue** a credit once. An operator can inspect each result, retry failed work and export the ledger.

Scope at a glance

12 fictional schools, 3 fictional providers and 14 days of seeded readings. Kenya, Nepal and Rwanda are example country labels. All measurements and outcomes are synthetic; country does not affect eligibility. Credits have no monetary value. This is an independent prototype, without UNICEF or Giga affiliation or approval.

01 / HOW IT WORKS

Follow one reading from input to result



React console: submit, inspect, retry and export

PostgreSQL also stores the reading, durable queue records and policy versions.

Receive. A measurement contains a school, reporting date, internet performance values and a simulated verification status. The API validates it and saves the reading plus a durable event record.

Queue. An outbox is a database-backed list of messages waiting to be sent. The producer sends these to real Apache Kafka. The consumer records receipt in PostgreSQL before processing.

Decide. The worker evaluates the current policy. It saves which version was used, the result of each check, and whether a credit was issued.

Record. The decision, any credit and the audit entry commit together. The Kafka offset, which records consumption progress, is acknowledged after durable handling.

A simple example

For a school day that has not already received a credit, this reading passes the illustrative default policy:

CONDITION	DEMO POLICY	EXAMPLE READING
Download speed	At least 20 Mbps	30 Mbps - pass
Upload speed	At least 5 Mbps	8 Mbps - pass
Latency (response delay)	At most 100 ms	60 ms - pass
Uptime (availability)	At least 98%	99% - pass
Verification	Marked verified	Verified - pass

Result: 10 demo credits, once

If every check passes, one credit record is added for that school and date. Another eligible reading for the same school day does not create a second issuance. These thresholds, the daily period and the amount are prototype assumptions, not requirements specified by UNICEF.

02 / TECHNICAL SPECIFICATION

A TypeScript application with durable state

The browser talks to a REST/JSON API. PostgreSQL holds the lasting records; Kafka carries events between ingestion and processing. The worker can be triggered from the console or run through a separate command-line entry point.

COMPONENT	IMPLEMENTED WITH	RESPONSIBILITY
User interface	React 18.3 + TypeScript Vite 6.4	Overview, schools, readings, ledger, events and policy.
Backend API	Node.js 24.19 Express 5.1 + TypeScript	Validated endpoints, business rules and worker orchestration.
Validation	Zod 3.25	Reject invalid payloads and return structured JSON errors.
Database	PostgreSQL 16.2 pg 8.16	Transactions, unique constraints, policies and durable event state.
Event streaming	Apache Kafka 4.4 Confluent Node client 1.10	Real producers and consumers, delivery acknowledgement and retries.
Packaging	Dockerfile + Compose	App, PostgreSQL and Kafka services. Configuration supplied; container execution not yet verified.
Automation	GitHub Actions configuration	Build and test pipeline supplied; no remote CI run claimed.

The main records

Schools and providers identify the subject of a reading. **Measurements** hold submitted values. **Rules** hold policy versions. **Events** track queued, processed or failed work. **Decisions** preserve checks and policy snapshots. **Credits** form the ledger; **activity** records the audit trail.

The API surface

Submit: POST /api/measurements
Inspect: GET /api/measurements/:id and GET /api/credits
Process or recover: POST /api/worker/run and POST /api/events/:id/retry
Change policy: PUT /api/rules | **Check health:** GET /api/health

Current local entry points: browser 5173; API/built app 3000; PostgreSQL 5433; Kafka 19092. Ports are development settings, not architectural requirements. The earlier Python implementation is inactive.

03 / RELIABILITY AND EVIDENCE

Make retries safe and decisions explainable

An event can arrive more than once, a service can disconnect, or processing can fail halfway through. The prototype makes these cases visible and checks that they do not produce duplicate credits.

SITUATION	EXPECTED BEHAVIOR
The same submission arrives again	The pair (source, external_id) identifies it. An identical payload returns the existing record; a conflicting payload returns HTTP 409.
Two readings qualify for one school day	A database uniqueness constraint permits one credit issuance for (school_id, period), including across policy versions.
The worker fails after inserting a credit	The transaction rolls back the credit, decision and audit changes together. Failure bookkeeping is saved; an operator can retry.
Kafka is unavailable	Kafka mode reports a service error and preserves queued work. It does not silently switch to a different transport.
The policy changes	New evaluations use the new version. Earlier decision snapshots keep their original checks and credit outcome.

Idempotency, in plain language

Repeating the same operation has no additional credit effect. Delivery can happen more than once; PostgreSQL constraints and transactions protect the result. This is not a claim of distributed exactly-once delivery.

What has been checked

Baseline validation passed **12 native backend tests** and **40 independent API, database and Kafka tests** against PostgreSQL and Kafka. Checks include overlapping requests, conflicting submissions, rollback, retry, policy history, broker outages and data migration.

Browser checks cover nine groups on desktop and mobile: submission, processing, failure/retry, policy editing, country/search filters, CSV export and reset behavior. The latest revision passed 14 focused backend checks, two independent API checks and all nine browser groups, with refreshed screenshots. Detailed commands and evidence are in the repository validation record.

Limits of this evidence

These local checks do not establish production capacity or real school impact. Seed decisions are precomputed; new and queued readings exercise Kafka. The broker has one partition and one replica; workers use manual assignment and database locks.

04 / REVIEW AND NEXT STEPS

A short walkthrough, with clear boundaries

A three-minute demonstration

1. **Overview:** explain that the figures are synthetic and show the live Kafka status.
2. **Clean batch:** open a new reading, inspect the five checks and find its credit in the ledger.
3. **Duplicate delivery:** show repeated delivery producing one issuance.
4. **Worker failure:** show the failed event, retry it and inspect the single resulting credit.
5. **Policy:** change one threshold and explain why previous decisions remain unchanged.

How this relates to the consultancy

TOR AREA	EVIDENCE IN THIS PROTOTYPE
Backend services and relational data	TypeScript/Express API, PostgreSQL schema and explicit validation.
Kafka, rules and workflow handling	Real broker, versioned rules, visible event states, retry and replay protection.
Quality and documentation	Automated checks, reproducible demo scenarios and operational README.
Deployment and operations	Health endpoint and container/CI configuration; production rollout remains to be done.
Security and integrations	Input validation and a limited-privilege DB role exist. Application authentication, partner connectors and value transfer remain outside this demo.

What comes next

Agree the architecture, data contracts and acceptance criteria with the product and tech team. Replace synthetic inputs with reviewed measurement/verification adapters; add authenticated roles, monitoring, backup/restore checks and deployment automation; test load and recovery before production use.

TypeScript is within the ToR language requirements. Kafka is named in its event-flow deliverable. Express, React, Vite and PostgreSQL are implementation choices for this prototype; the ToR does not mandate them. No separate rules/workflow-engine product or Web3 integration is claimed.

Sources and review material

Basis: supplied Connectivity Credits Backend Software Engineering Consultancy Terms of Reference, especially pp. 1-2 (purpose and event processing), p. 4 (deliverables) and p. 5 (technical experience). Vacancy: jobs.unicef.org/en-us/job/596538.
Implementation evidence: README.md, VALIDATION.md, server/src and artifacts/browser-kafka in the companion project. Prepared 30 October 2026. This document describes the local prototype; no public demo URL is claimed.